



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 10

“Teclado Matricial”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 27 de febrero de 2025

INTRODUCCIÓN

Teclado matricial

Un teclado matricial es un dispositivo de entrada diseñado para ahorrar una gran cantidad de pines en un microcontrolador. En lugar de usar un pin para cada botón, los botones se organizan en filas y columnas.

Estructura y ahorro de pines

Conexión directa: Se necesitan 16 pines del microcontrolador más un pin común (17 en total).

Conexión matricial: Solo se necesitan 8 pines (4 para las filas y 4 para las columnas).

Internamente, cada tecla es un interruptor que, al ser presionado, conecta una fila específica con una columna específica.

Método de "Barrido" (Scanning)

Dado que el microcontrolador no puede "ver" todas las teclas a la vez, realiza un proceso cíclico y muy rápido llamado barrido:

1. **Salida (Filas):** El microcontrolador pone todas las filas en un estado lógico "Alto" (1), excepto una, que pone en "Bajo" (0).
2. **Lectura (Columnas):** El microcontrolador lee los pines de las columnas. Si una columna se lee como "Bajo", significa que el botón en la intersección de esa fila y esa columna está presionado.
3. **Rotación:** El proceso se repite para la siguiente fila. Esto sucede miles de veces por segundo, por lo que para el humano parece instantáneo.

Consideraciones de diseño

Para que un teclado matricial funcione correctamente en un entorno real, hay que tener en cuenta:

- **Resistencias Pull-up:** Se necesitan resistencias (internas o externas) para asegurar que, cuando no se presiona nada, los pines de lectura tengan un valor definido y no floten captando ruido.
- **Anti-rebote (Debounce):** Al igual que los botones individuales, las teclas de la matriz rebotan mecánicamente. El software debe esperar unos milisegundos antes de confirmar que una tecla fue presionada.
- **Detección de Teclas Múltiples:** Si presionas tres teclas a la vez en una matriz simple, puede ocurrir un efecto llamado "fantasma" (ghosting), donde el microcontrolador cree que se presionó una cuarta tecla que no existe.

Implementación

CodeVisionAVR facilita mucho su implementación porque incluye una librería estándar llamada `keypad.h` (o similar dependiendo de la versión).

En el CodeWizardAVR, se puede habilitar el soporte para teclados matriciales, indicar en qué puerto está conectado y el software generará la función que te devuelve directamente el carácter presionado ('1', 'A', '*', etc.).

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 10, el circuito a armar es el siguiente:

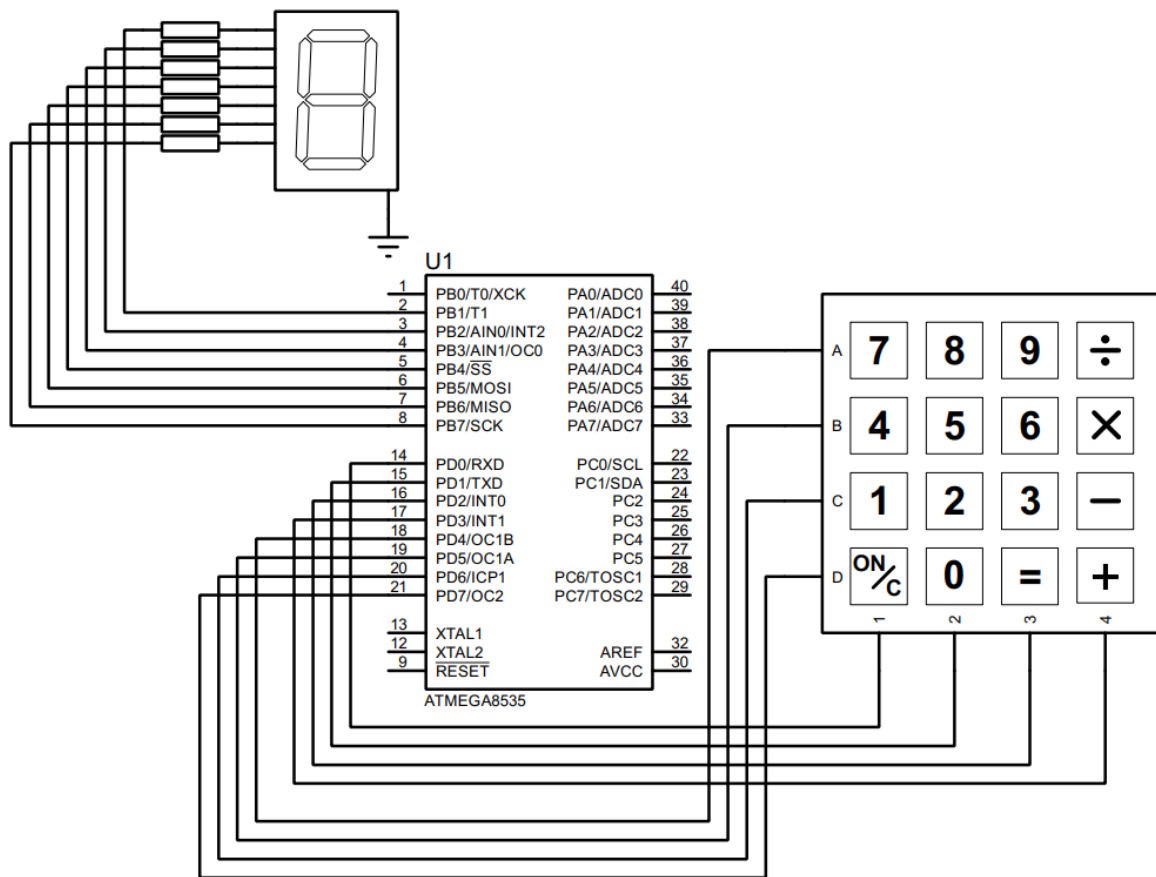


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, el registro B se configura como de salida (PB1 – PB7), mientras que el puerto D, la mitad como de salida (PD0 – PD3) y la otra como de entrada (PD4 – PD7), donde en este caso no se activan las resistencias de pull-up.

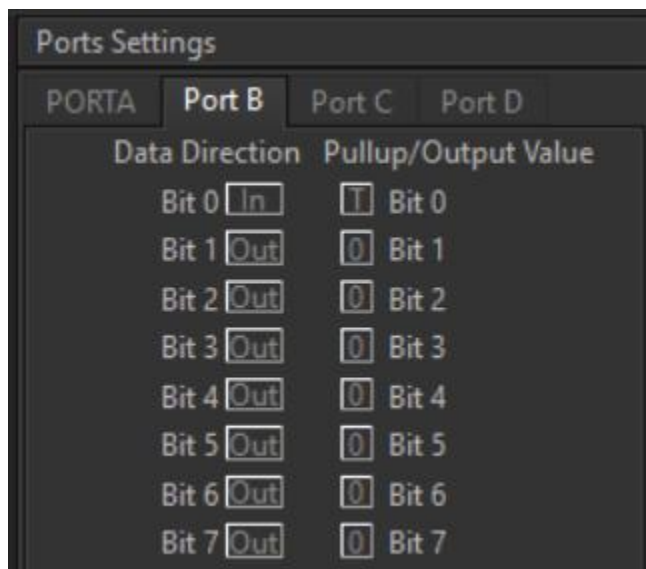


Imagen 2. Configuración del puerto B.

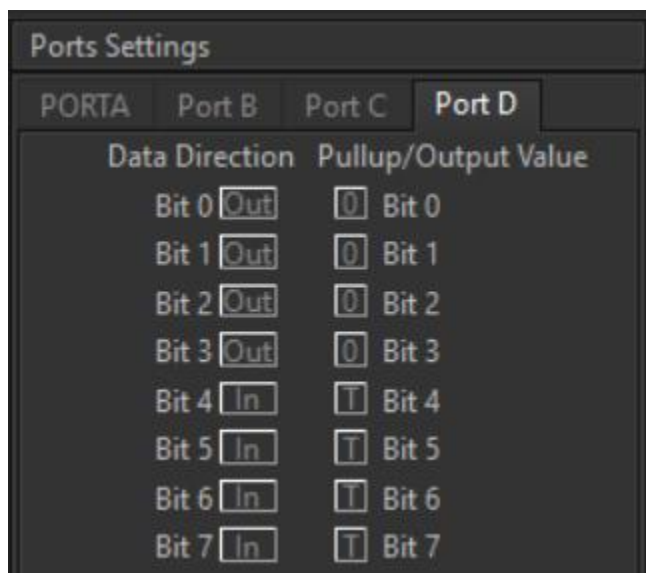


Imagen 3. Configuración del puerto D.

Para poder hacer la representación de los botones del teclado matricial en el display como número hexadecimal, para ello se realizó la siguiente tabla, tomando en cuenta lo que se indico en la práctica 3.

Para esta única práctica, el presionar la tecla “*” y “#”, en el display se mostrará un guion medio y si ninguna tecla se presiona, entonces un guion bajo. Es por esa razón que su equivalencia en hexadecimal es agregada a la tabla.

Otro cambio importante fue que debido a que el teclado matricial solo posee las letras de la ‘A’ a la ‘D’, las letras de la ‘E’ a la ‘G’ se descartan de la tabla.

N��mero Display	Combinaciones								Valor hexadecimal
	g	f	e	d	c	b	a		
0	0	1	1	1	1	1	1	0	0x7e
1	0	0	0	0	1	1	0	0	0x0c
2	1	0	1	1	0	1	1	0	0xb6
3	1	0	0	1	1	1	1	0	0x9e
4	1	1	0	0	1	1	0	0	0xcc
5	1	1	0	1	1	0	1	0	0xda
6	1	1	1	1	1	0	1	0	0xfa
7	0	0	0	0	1	1	1	0	0x0e
8	1	1	1	1	1	1	1	0	0xfe
9	1	1	0	1	1	1	1	0	0xde
A	1	1	1	0	1	1	1	0	0xee
B	1	1	1	1	1	0	0	0	0xf8
C	0	1	1	1	0	0	1	0	0x72
D	1	0	1	1	1	1	0	0	0xbc
-	1	0	0	0	0	0	0	0	0x80
_	0	0	0	1	0	0	0	0	0x10

Tabla 1. Representaci  n hexadecimal de los botones del teclado matricial.

C  digo del circuito de CodeVision:

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4       Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr  ctica 10 'Teclado Matricial'
8  Version : 1.0
9  Date    : 23/02/2026
10 Author  : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model        : Small
19 External RAM size    : 0
20 Data Stack size      : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>

```

```

27 // Declare your global variables here
28
29 void main(void) {
30     // Declare your local variables here
31     unsigned char tecla, lectura;
32     const char keypad[16] = {
33         0x7e, 0x0c, 0xb6, 0x9e, // 0 1 2 3
34         0xcc, 0xda, 0xfa, 0xe, // 4 5 6 7
35         0xfe, 0xde, 0xee, 0xf8, // 8 9 A B
36         0x72, 0xbc, 0x80, 0x10 // C D - _
37     };
38
39     // Input/Output Ports initialization
40     // Port A initialization
41
42
43     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
44     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
45     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
46     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
47
48     // Port B initialization
49     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
50     DDRB=(1<<DDB7) | (1<<DDB6) | (1<<DDB5) | (1<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) | (0<<DDB0);
51     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=T
52     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

```

```

54     // Port C initialization
55     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
56     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
57     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
58     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
59
60     // Port D initialization
61     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=Out Bit2=Out Bit1=Out Bit0=Out
62     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (1<<DDD3) | (1<<DDD2) | (1<<DDD1) | (1<<DDD0);
63     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=0 Bit2=0 Bit1=0 Bit0=0
64     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);
65
66     // Timer/Counter 0 initialization
67     // Clock source: System Clock
68     // Clock value: Timer 0 Stopped
69     // Mode: Normal top=0xFF
70     // OC0 output: Disconnected
71     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
72     TCNT0=0x00;
73     OCR0=0x00;

```

```

75     // Timer/Counter 1 initialization
76     // Clock source: System Clock
77     // Clock value: Timer1 Stopped
78     // Mode: Normal top=0xFFFF
79     // OC1A output: Disconnected
80     // OC1B output: Disconnected
81     // Noise Canceler: Off
82     // Input Capture on Falling Edge
83     // Timer1 Overflow Interrupt: Off
84     // Input Capture Interrupt: Off
85     // Compare A Match Interrupt: Off
86     // Compare B Match Interrupt: Off
87     TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
88     TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
89     TCNT1H=0x00;
90     TCNT1L=0x00;
91     ICR1H=0x00;
92     ICR1L=0x00;
93     OCR1AH=0x00;
94     OCR1AL=0x00;
95     OCR1BH=0x00;
96     OCR1BL=0x00;

```



```

98 // Timer/Counter 2 initialization
99 // Clock source: System Clock
100 // Clock value: Timer2 Stopped
101 // Mode: Normal top=0xFF
102 // OC2 output: Disconnected
103 ASSR=0<<AS2;
104 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
105 TCNT2=0x00;
106 OCR2=0x00;
107
108 // Timer(s)/Counter(s) Interrupt(s) initialization
109 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
110
111 // External Interrupt(s) initialization
112 // INT0: Off
113 // INT1: Off
114 // INT2: Off
115 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
116 MCUCSR=(0<<ISC2);
117
118 // USART initialization
119 // USART disabled
120 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```

```

122 // Analog Comparator initialization
123 // Analog Comparator: Off
124 // The Analog Comparator's positive input is
125 // connected to the AIN0 pin
126 // The Analog Comparator's negative input is
127 // connected to the AIN1 pin
128 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
129 SFIOR=(0<<ACME);
130
131 // ADC initialization
132 // ADC disabled
133 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
134
135 // SPI initialization
136 // SPI disabled
137 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
138
139 // TWI initialization
140 // TWI disabled
141 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```

```

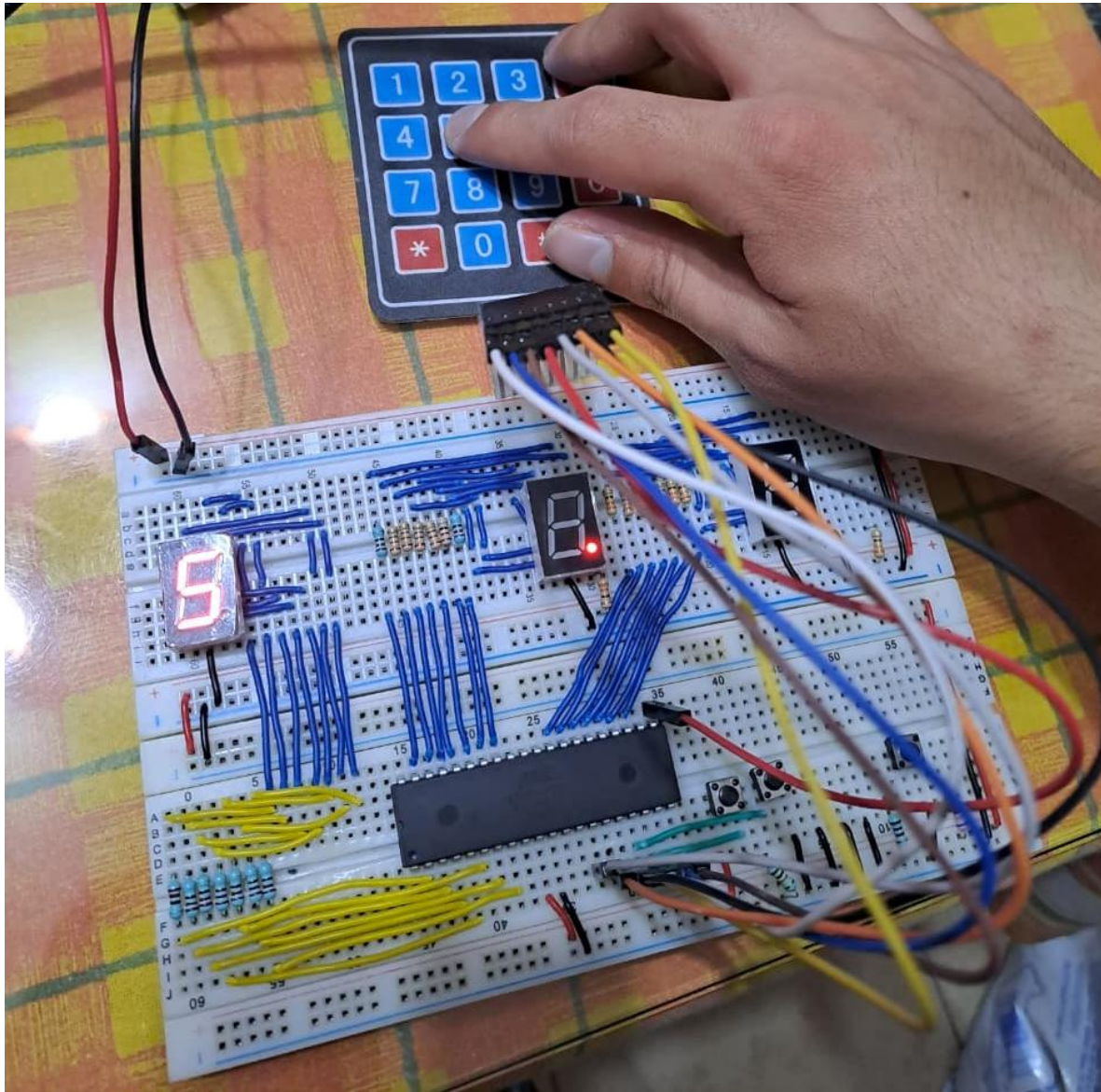
143 while (1) {
144     // Place your code here
145     // Se muestra un guiñe bajo por defecto
146     tecla = 15;
147
148     // Activa la primera columna de la izquierda
149     PORTD = 0b11111110;
150     delay_ms(1);
151
152     // Desplazamos 4 bits a la derecha
153     // solo nos interesa el valor de los bits
154     lectura = PIND >> 4;
155
156     // Leemos el valor de los bits después del desplazamiento
157     switch(lectura) {
158         case 0b1110: tecla = 1; break;
159         case 0b1101: tecla = 4; break;
160         case 0b1011: tecla = 7; break;
161         case 0b0111: tecla = 14; break;
162     }
163
164     // Activa la segunda columna de la izquierda
165     PORTD = 0b11111101;
166     delay_ms(1);

```

```
168 // Desplazamos 4 bits a la derecha
169 // solo nos interesa el valor de los filas
170 lectura = PIND >> 4;
171
172 // Leemos el valor de las filas después del desplazamiento
173 switch(lectura) {
174     case 0b1110: tecla = 2; break;
175     case 0b1101: tecla = 5; break;
176     case 0b1011: tecla = 8; break;
177     case 0b0111: tecla = 0; break;
178 }
179
180 // Activa la tercera columna de la izquierda
181 PORTD = 0b11111011;
182 delay_ms(1);
183
184 // Desplazamos 4 bits a la derecha
185 // solo nos interesa el valor de los filas
186 lectura = PIND >> 4;
187
188 // Leemos el valor de las filas después del desplazamiento
189 switch(lectura) {
190     case 0b1110: tecla = 3; break;
191     case 0b1101: tecla = 6; break;
192     case 0b1011: tecla = 9; break;
193     case 0b0111: tecla = 14; break;
194 }
```

```
196 // Activa la cuarta columna de la izquierda
197 PORTD = 0b11110111;
198 delay_ms(1);
199
200 // Desplazamos 4 bits a la derecha
201 // solo nos interesa el valor de los filas
202 lectura = PIND >> 4;
203
204 // Leemos el valor de las filas después del desplazamiento
205 switch(lectura) {
206     case 0b1110: tecla = 10; break;
207     case 0b1101: tecla = 11; break;
208     case 0b1011: tecla = 12; break;
209     case 0b0111: tecla = 13; break;
210 }
211
212 // Se actualiza el display
213 PORTB = keypad[tecla];
214 delay_ms(200);
215 }
216 }
```


Circuito físico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

En esta práctica vio el funcionamiento del teclado matricial, que no es nada más que un arreglo de teclas en forma de matriz de filas y columnas. Esto nos permite detectar muchas teclas usando menos pines del ATMEGA.

Cada una de las teclas es un botón que conecta una fila con una columna cuando es presionado. En este caso las filas se configuraron como de entrada y las columnas como de salida y como son cuatro filas y cuatro columnas, se usó todo el puerto D, conectando las

filas a los cuatro bits más significativos y las columnas a los cuatro menos significativos, por supuesto configurándolos como pines de entrada y de salida respectivamente.

Con esta configuración lo único que se hace es activar una columna a la vez y leer el valor de las filas en dicha columna al presionarse un botón. Para esto se lee todo el puerto D, pero haciendo un corrimiento de cuatro bits hacia la derecha puesto que el valor de las filas se refleja en los 4 bits más significativos, esto nos ayuda a leer solo los cuatro bits menos significativos y asignarles el índice correspondiente al arreglo que muestra su valor en el display de 7 segmentos.

Al final lo más importante de esta práctica es la introducción a las matrices de una forma básica que será de ayuda para futuras prácticas donde se usan matrices de leds de dimensiones más grandes, que tienen un funcionamiento muy similar a lo visto en esta práctica.

- **Meléndez Macedonio Rodrigo**

Para el desarrollo de esta práctica utilizamos el teclado matricial, el cual nos sirvió para crear un circuito funcional en el que podemos interactuar con el teclado matricial presionando sus botones para generar un valor en un display de 7 segmentos con de acuerdo con el valor que indica en cada botón.

Este tipo de teclados funcionan como una matriz la cual contiene filas y columnas, haciendo que su uso sea más simple, pues la cantidad de pines es menor. Teniendo eso en cuenta, usamos CodeVisionAVR para configurar todo el puerto D para que leyera las filas como entradas y las columnas como salidas, haciendo un corrimiento de 4 bits hacia la derecha para asignar un índice al arreglo que muestra el valor en el display de 7 segmentos. Entender el funcionamiento del teclado matricial es vital, pues su uso básico en esta práctica nos ayudará el desarrollo de futuras prácticas usando este tipo de dispositivos.

Adicionalmente, es importante aclarar que en nuestro caso configuramos el teclado matricial de manera que al momento que solo permitirá representar números del 0 al 9 en el display de 7 segmentos, por lo cual, los otros valores que tiene el teclado son ignorados y al momento de presionar dichos botones solo aparece un guión "-" representado en el display.

BIBLIOGRAFÍA

(s.f.). *Keypad Scanning*. All About Circuits. Recuperado el 7 de marzo de 2026. <https://www.allaboutcircuits.com/>

(s.f.). *How to set up a keypad*. Circuits Basics. Recuperado el 7 de marzo de 2026. <https://www.circuitbasics.com/how-to-set-up-a-keypad-on-an-arduino/>